

Practical Tasks in Matrix and Tensor Algorithms

Revised assignment specification: explicit requirements, inputs, outputs, tests, and quality criteria

Purpose of the document

Each task is formulated so that students can immediately write code, a README, and a report. For Tasks 2, 3, and 4, the interface of a functional matrix is specified separately: the algorithm must not build the entire matrix in advance, except for the reference SVD on small problem sizes.

Reference packs added to each task

How to use the references. Each task now has a short reference pack placed immediately after the task statement. The packs are designed as a zero-knowledge path: first learn the required mathematical objects and notation, then read the algorithmic source, then use the implementation-oriented paper or chapter for tests and edge cases.

Page ranges refer to journal/book pagination where available. For books with multiple editions or PDF front matter, also use the chapter, section, or algorithm names listed in the bullet.

Contents

- General rules for all tasks
- Input formats: functional matrices and tensor oracles
- Matrix algorithms: Tasks 1-7
- Tensor algorithms: Tasks 8-13
- Reference packs attached to Tasks 1-13
- Recommended repository and report structure
- Pre-submission checklist

1. General rules for all tasks

Practicum objective. Implement basic matrix and tensor algorithms for low-rank approximation, compare them on shared tests, and learn to control result quality through errors, ranks, number of data accesses, and runtime.

Language and libraries. The assignment is not tied to a specific programming language. Unless the instructor requires otherwise, Python 3.10+ with NumPy/SciPy is convenient for linear algebra, and PyTorch/JAX is convenient for the neural-network task. Ready-made implementations of the algorithms being studied must not be used. Basic BLAS/LAPACK operations are allowed: matrix multiplication, QR, SVD of small matrices, and solving systems of linear equations.

Unified API style. Each implementation must be designed as a function or class with clear inputs, outputs, accuracy parameters, and a maximum number of iterations. Every randomized algorithm must accept `random_seed`.

- For each algorithm, prepare at least 3 tests: a simple analytical example, a random synthetic example, and an example with noise or poor conditioning.
- In all experiments, report relative error, runtime, number of iterations, actual rank, and the number of accesses to elements, rows, columns, or tensor slices when applicable.
- For small problem sizes, run a reference full SVD or direct error computation. For large problem sizes, estimate the error on a sample.
- The report must include not only final numbers, but also a short comment: where the method worked, where it did not, and why.
- When parameters such as `r`, `eps`, `oversampling`, or `max_iter` are present, show in the report how the result changes when these parameters vary.

2. Unified quality metrics

Main error. If the full matrix or tensor X is available, use the relative Frobenius-norm error: the Frobenius norm of $(X \text{ minus its approximation})$ divided by the Frobenius norm of X . For matrices, it is also useful to compute the relative spectral-norm error using SVD on small problem sizes.

Sample-based errors. For functional matrices and tensor oracles where the full object must not be built, choose a random set of indices Ω and compute RMSE or relative error on Ω .

Method cost. Separately measure runtime, the number of element-oracle calls, the number of computed columns, rows, or slices, and the memory used by the factors.

Metric	Formula / meaning	Where to use	Comment
rel_frob	Frobenius norm of error / Frobenius norm of object	all full small tests	main approximation metric
rel_spec	spectral norm of error / spectral norm of matrix	small matrix tests	compare with the SVD tail
sample_rmse	square root of mean sampled squared error	functional objects	mandatory for large tasks
oracle_calls	number of requests to $A(i,j)$ or $X(i_1, \dots, i_d)$	cross methods, TT-cross, rSVD-oracle	shows the cost of data access
compression	dense-object memory / factor memory	all low-rank methods	shows practical usefulness

3. Input formats

3.1. Functional matrix for Tasks 2, 3, and 4

Definition. A functional matrix is an $m \times n$ matrix A whose elements are computed by a function rather than stored as a dense array. The algorithm must work through function queries and must not build all of A , except in small reference tests.

```
class FunctionalMatrix:
    shape: tuple[int, int]
    dtype: float | complex

    def entry(i: int, j: int) -> scalar:
        # return A[i, j]

    def block(I: array_like, J: array_like) -> ndarray:
        # return the submatrix A[np.ix_(I, J)]

    def row(i: int, J=None) -> ndarray:
        # return row A[i, :] or A[i, J]

    def col(I, j: int) -> ndarray:
        # return column A[:, j] or A[I, j]

    # For rSVD it is preferable to support products without explicitly building A.
    def matvec(x) -> ndarray: # A @ x
    def rmatvec(y) -> ndarray: # A.T @ y
```

Minimum set of test functional matrices. Use at least: an exact low-rank matrix $A = U V^T$; a Hilbert matrix $A_{ij} = 1/(i+j+1)$; a Cauchy/kernel matrix $A_{ij} = 1/(x_i + y_j)$ or $\exp((x_i y_j)^2 / \sigma^2)$; and a low-rank matrix with added noise.

- In Tasks 2, 3, and 4, the full array A must not be built inside the algorithm.
- In the `evaluate_small` reference function, the full A may be built only for m, n not exceeding a fixed threshold, for example 1000.
- The README must explicitly state which oracle methods are actually used by each algorithm.
- If `matvec/rmatvec` are unavailable, rSVD can be implemented through block requests, but this must be described separately.

3.2. Tensor inputs for Tasks 8-13

Dense tensor. An array X of size $n_1 \times \dots \times n_d$ that is fully available in memory. It is used in Tasks 9, 11, and 12 on small or medium problem sizes.

Canonical CP format. A tensor is represented as a sum of R rank-1 tensors: $X[i_1, \dots, i_d] = \sum_{a=1}^R \lambda_a \cdot \prod_k U_k[i_k, a]$. Store the weights λ and the factor matrices U_k .

Tensor oracle. For TT-cross, the tensor is represented by a function `entry(i1, ..., id)` that returns a single element. For large problem sizes, the full tensor must not be built.

```
class FunctionalTensor:
    shape: tuple[int, ...]

    def entry(indices: tuple[int, ...]) -> scalar:
        # return X[i1, ..., id]

    def batch(indices: ndarray) -> ndarray:
```

4. Matrix algorithms

1. Lanczos method

Goal. Implement the iterative Lanczos method for a symmetric matrix or self-adjoint linear operator and use it to approximate extreme eigenvalues.

Input data. A matrix A of size $n \times n$ or a function $\text{matvec}(x) = Ax$. In the basic version, assume A is real symmetric. Input parameters: number of steps k , initial vector q_0 , stopping tolerance, and a flag for full or partial reorthogonalization.

Required implementation:

- Construct an orthonormal basis Q_k and a tridiagonal matrix T_k with diagonal α and subdiagonal β .
- Handle happy breakdown: if β becomes smaller than tolerance, stop the iterations correctly.
- Add optional reorthogonalization to combat loss of orthogonality.
- Compute Ritz values: eigenvalues of T_k used as approximations to eigenvalues of A .
- Provide an interface that works with matvec , so storing A is not required.

Checks and experiments:

- Compare extreme eigenvalues with `numpy/scipy.linalg.eigh` on small n .
- Plot the error of the maximum/minimum eigenvalue against the number of iterations k .
- Check orthogonality of Q_k : $\|Q_k^T Q_k - I\|_F$.
- Test the method on a diagonal matrix with known spectrum and on a random symmetric matrix.

Submit:

- Function `lanczos(matvec, n, k, q0=None, tol=..., reorthogonalize=False)`.
- Tests and a table with Ritz value errors.
- A short description of how reorthogonalization affects the result.

Acceptance criteria:

- There must be no access to $A[i,j]$ if only matvec is supplied.
- On symmetric tests, the method should show monotonic improvement at least for the largest eigenvalue as k grows.
- The code must work correctly under early stopping.

Optional extensions:

- Add residual estimates $\|Ay - \theta y\|$ for Ritz pairs.
- Support the search for several extreme eigenpairs.

Reference pack for Task 1

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Saad, Y. *Numerical Methods for Large Eigenvalue Problems*, 2nd ed. (SIAM, 2011). Read Ch. 1 for the eigenvalue problem, Ch. 4 for projection/Ritz ideas, then Ch. 6, pp. 125-171; the Lanczos algorithm itself is Algorithm 6.5 on pp. 146-147, with convergence and residual discussion on pp. 156-171. This covers Krylov subspaces, Ritz values/pairs, tridiagonalization, happy breakdown, and reorthogonalization motivation.
- Implementation support: Bai, Z., Demmel, J., Dongarra, J., Ruhe, A., and van der Vorst, H. *Templates for the Solution of Algebraic Eigenvalue Problems* (SIAM, 2000), pp. 60-87. Use this for practical stopping criteria, residual norms, orthogonality loss, and comparison with Arnoldi/Lanczos variants.

2. Matrix cross approximation for arbitrary rank r

Goal. Implement low-rank approximation of a functional matrix using cross/skeleton approximation for a specified rank r .

Input data. A functional matrix A of size $m \times n$, available through `entry`, `block`, `row`, and `col`. Parameters: desired rank r , initial row/column indices or a rule for selecting them, and `max_sweeps` for pivot improvement.

Required implementation:

- Construct an approximation $\hat{A} = C M R$, where $C = A[:,J]$, $R = A[I,:]$, and $M = (A[I,J])^{-1}$ or the pseudoinverse for poor conditioning.
- Support $r > 1$: store row and column index sets I and J of length r , not just a single pivot.
- Implement pivot selection: random/greedy initial choice and improvement through `maxvol` or a simplified greedy residual search.
- Do not build the full A inside the algorithm. Only rows, columns, and blocks requested from `FunctionalMatrix` are allowed.

- Return factors C, M, R and the selected index lists I, J.

Checks and experiments:

- On an exact rank-r matrix, verify that the error is close to machine precision when pivots are chosen successfully.
- Compare the error with the best rank-r SVD approximation on small sizes.
- Show error as a function of r for Hilbert, Cauchy, and Gaussian-kernel matrices.
- Count the number of requested elements, rows, and columns.

Submit:

- Function `cross_rank_r(Afun, r, init="random|maxvol|greedy", tol=None)`.
- A result object with methods `reconstruct_small()`, `matvec(x)`, and `entry(i,j)`.
- Table: r, rel_frob, sample_rmse, oracle_calls, time.

Acceptance criteria:

- The algorithm must actually work with a functional matrix.
- The $r=1$ case must be a special case of the general r implementation, not a separately hard-coded algorithm.
- If $A[I,J]$ is poorly conditioned, use `pinv` or regularization and issue a warning.

Optional extensions:

- Implement maxvol-like improvement of selected rows/columns.
- Add a block version to reduce the number of single entry queries.

Reference pack for Task 2

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Goreinov, S. A., Tyrtshnikov, E. E., and Zamarashkin, N. L. "A theory of pseudoskeleton approximations." *Linear Algebra and its Applications* 261(1-3), 1997, pp. 1-21. Use pp. 1-7 for the idea of approximating a matrix by selected rows and columns; pp. 8-18 for accuracy and submatrix-volume arguments.
- Practical ACA/cross implementation: Bebendorf, M., and Rjasanow, S. "Adaptive Low-Rank Approximation of Collocation Matrices." *Computing* 70, 2003, pp. 1-24. Use pp. 1-7 for why only rows/columns are queried; pp. 8-17 for the adaptive pivot logic and stopping; pp. 18-24 for numerical behavior.
- Pivot/maxvol bridge to tensor cross: Oseledets, I. V., and Tyrtshnikov, E. E. "TT-cross approximation for multidimensional arrays." *Linear Algebra and its Applications* 432(1), 2010, pp. 70-88, especially pp. 72-78 for skeleton/cross interpolation ideas and pp. 78-82 for maxvol-style index selection.

3. Randomized SVD for functional matrices

Goal. Implement randomized SVD as a method for constructing an approximate low-rank decomposition $A \approx U S V^T$ using only operations with a functional matrix.

Input data. A functional matrix A of size $m \times n$. The basic version requires `matvec/rmatvec` or block products $A @ \Omega$ and $A^T @ Y$. Parameters: rank r, oversampling p, number of power iterations q, and `random_seed`.

Required implementation:

- Generate a random matrix Ω of size $n \times (r+p)$ and form $Y = A \Omega$.
- Find an orthonormal basis $Q = \text{orth}(Y)$ using QR.
- If $q > 0$, perform power iterations: $Y = A(A^T Q)$, with orthonormalization at each step.
- Construct the small matrix $B = Q^T A$ and compute $\text{SVD}(B)$.
- Return $U = Q U_B$, S, and V^T .
- Do not build the full A except in small reference tests.

Checks and experiments:

- Compare with exact SVD for small matrices: error and singular values.
- Show the effect of oversampling p and power iterations q.
- Test on matrices with rapidly and slowly decaying singular values.
- Compare with the cross approximation from Task 2 for the same r.

Submit:

- Function `randomized_svd(Afun, r, p=10, q=0, seed=...)`.
- Experiment log with a table of rel_frob, rel_spec, time, and `matvec_count`.
- A short conclusion: when q helps and when it only increases the cost.

Acceptance criteria:

- U and V must be orthonormal to tolerance.

- As r increases, the error should decrease on average.
- For an exact rank- r matrix, the method should recover it with small error when oversampling is sufficient.

Optional extensions:

- Add a single-pass variant when access to the matrix is expensive.
- Add error estimation without fully building A .

Reference pack for Task 3

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Halko, N., Martinsson, P. G., and Tropp, J. A. "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions." *SIAM Review* 53(2), 2011, pp. 217-288. Start with pp. 217-229 for low-rank approximation and randomized range finding; use pp. 230-244 for basic randomized SVD algorithms, oversampling, and power iterations; use pp. 245-275 for error estimates.
- Matrix approximation baseline: Eckart, C., and Young, G. "The approximation of one matrix by another of lower rank." *Psychometrika* 1(3), 1936, pp. 211-218. Use this as the mathematical basis for comparing randomized SVD with the best rank- r SVD approximation.

4. Adaptive cross approximation with caching for functional matrices

Goal. Implement an adaptive cross method that selects pivots automatically and reuses previously computed elements. A reliable rank-1 version is minimally sufficient, but the architecture should allow extension to $r > 1$.

Input data. A functional matrix A of size $m \times n$. Parameters: eps , max_rank , max_sweeps , initial indices, and $\text{cache}=\text{True}$. In the minimal rank-1 implementation, build $\hat{A} = u v^T / \text{pivot}$.

Required implementation:

- Introduce a query cache: repeated calls to $\text{entry}(i,j)$, $\text{row}(i)$, or $\text{col}(j)$ must not recompute already known elements.
- For $r=1$, choose a pivot (i^*, j^*) with sufficiently large $|A[i^*, j^*]|$, construct $u = A[:, j^*]$, $v = A[i^*, :]$, and $\hat{A} = u v^T / A[i^*, j^*]$.
- For adaptivity, estimate the residual on a random or special validation sample of indices.
- If the error exceeds eps , add the next pivot to the sum of rank updates or at least report that $r=1$ is insufficient.
- For the extended version, implement sequential rank updates on the residual: $A_{k+1} = A_k + \text{cross}(\text{residual})$.

Checks and experiments:

- Verify that the cache reduces the number of real calls to the original function during repeated requests.
- On a rank-1 matrix, achieve nearly zero error.
- On a rank > 1 matrix, show that $r=1$ gives a nonzero error and correctly diagnoses insufficient rank.
- Compare the result with Task 2 for $r=1$.

Submit:

- Class `CachedFunctionalMatrix` or wrapper `CountingFunctionalMatrix`.
- Function `adaptive_cross_cached(Afun, eps=..., max_rank=..., rank1_only=False)`.
- Pivot-selection log and a report of `cache_hits/cache_misses`.

Acceptance criteria:

- Repeated requests to the same element must not increase the counter of real computations.
- The pivot must not be zero; if a pivot is small, fall back to another index.
- For the rank-1 test, reconstruction must satisfy the specified eps .

Optional extensions:

- Implement full ACA/ACA+ with stopping by residual norm.
- Add a pivot-selection strategy based on the maximum residual on a validation grid.

Reference pack for Task 4

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Bebendorf, M., and Rjasanow, S. "Adaptive Low-Rank Approximation of Collocation Matrices." *Computing* 70, 2003, pp. 1-24. Use pp. 1-7 for low-rank matrix blocks and oracle-style row/column access; pp. 8-17 for adaptive pivoting and rank growth; pp. 18-24 for experiments.
- Earlier theory and kernel-matrix context: Bebendorf, M. "Approximation of boundary element matrices." *Numerische Mathematik* 86(4), 2000, pp. 565-589. Use pp. 565-574 for why smooth kernels admit low-rank blocks and pp. 575-589 for approximation/error behavior.

- Caching and post-processing companion: Bebendorf, M., and Kunis, S. "Recompression Techniques for Adaptive Cross Approximation." Journal of Integral Equations and Applications 21(3), 2009, pp. 331-357. Use pp. 331-340 for ACA factor storage and pp. 341-357 for recompression and efficiency.

5. Recompression of a low-rank decomposition

Goal. Implement recompression/rounding: transform an already computed low-rank decomposition to a smaller or more stable rank without significant loss of accuracy.

Input data. A matrix decomposition is supplied: $\hat{A} = U V^T$, $C M R$, or $U S V^T$. Parameters: ϵ or target rank r_{new} . The factor dimensions may be larger than the true numerical rank.

Required implementation:

- For $\hat{A} = U V^T$, compute QR decompositions $U = Q_U R_U$ and $V = Q_V R_V$, then compute the SVD of the small matrix $R_U R_V^T$.
- Truncate small singular values by ϵ or keep target_rank .
- Return compressed factors $U_c S_c V_c^T$ or $U_{\text{new}} V_{\text{new}}^T$.
- Preserve the ability to compute $\text{entry}(i,j)$ and $\text{matvec}(x)$ through the compressed factors.
- Handle cases where the original factors are poorly conditioned.

Checks and experiments:

- Create a deliberately redundant rank- $2r$ decomposition for a matrix of numerical rank r and show rank reduction.
- Compare the error before and after recompression.
- Check that factor memory decreases.
- Apply recompression to the results of Tasks 2 and 3.

Submit:

- Function `recompress_low_rank(factors,  $\epsilon$ =None, rank=None)`.
- Table: initial rank, new rank, error before/after, compression ratio.
- Explanation of the singular-value truncation rule.

Acceptance criteria:

- Recompression must not worsen the error by more than the stated ϵ .
- For an exact redundant decomposition, the correct numerical rank must be recovered.
- Operations must be performed on small matrices, not through the full A when only low-rank format is available.

Optional extensions:

- Add a variant for CUR/CMR by converting to $U V^T$.
- Implement automatic ϵ selection based on the relative tail of singular values.

Reference pack for Task 5

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Eckart, C., and Young, G. "The approximation of one matrix by another of lower rank." Psychometrika 1(3), 1936, pp. 211-218. This is the core theorem behind truncating small singular values with a controlled low-rank error.
- Algorithmic path: Bebendorf, M., and Kunis, S. "Recompression Techniques for Adaptive Cross Approximation." Journal of Integral Equations and Applications 21(3), 2009, pp. 331-357. Use pp. 331-340 for low-rank factor formats produced by ACA; pp. 341-350 for recompression by small dense linear algebra; pp. 351-357 for numerical examples.
- Broader randomized/SVD context: Halko, N., Martinsson, P. G., and Tropp, J. A. SIAM Review 53(2), 2011, pp. 217-288, especially pp. 230-244 and pp. 276-288 for small core factorizations and implementation patterns.

6. QR with column pivoting (pivoted QR)

Goal. Implement or correctly use QR decomposition with column pivoting for rank revelation and for constructing an interpolative matrix approximation.

Input data. Dense matrix A for the basic version. A variant is allowed where A is built from a functional matrix only in small tests. Parameters: tolerance or target_rank .

Required implementation:

- Compute $A P = Q R$, where P is a column permutation.
- Estimate numerical rank from the diagonal of R : $|R_{kk}| / |R_{11}| > \text{tolerance}$.
- Construct the rank- r approximation $\hat{A} = Q_r R_r P^T$.
- Return the pivot-column permutation and explain which columns are selected first.
- If `scipy.linalg.qr(..., pivoting=True)` is used, separately implement the wrapper, rank selection, and approximation construction.

Checks and experiments:

- Compare the rank found by pivoted QR with the numerical rank from SVD.
- Test matrices with linearly dependent and nearly linearly dependent columns.
- Plot the diagonal $|R_{kk}|$ and the singular values.
- Compare the QR-approximation error with the best SVD approximation of the same rank.

Submit:

- Function `pivoted_qr_approx(A, tol=None, rank=None)`.
- Pivot-index lists, Q , R , estimated rank, and $\hat{A}$.
- A mini-report on how much pivoted QR loses to SVD in error.

Acceptance criteria:

- Q must be orthonormal.
- The permutation P must correctly reconstruct $A P = Q R$.
- The rank-selection criterion must be explicitly described and reproducible.

Optional extensions:

- Construct interpolative decomposition based on pivoted QR.
- Add row pivoting or full pivoting for comparison.

Reference pack for Task 6

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Chan, T. F. "Rank Revealing QR Factorizations." *Linear Algebra and its Applications* 88-89, 1987, pp. 67-82. Use pp. 67-72 for why QR with pivoting reveals numerical rank; pp. 73-82 for factorization quality and rank decisions.
- Strong RRQR theory: Gu, M., and Eisenstat, S. C. "Efficient Algorithms for Computing a Strong Rank-Revealing QR Factorization." *SIAM Journal on Scientific Computing* 17(4), 1996, pp. 848-869. Use pp. 848-854 for definitions and failure modes of classical pivoting; pp. 855-869 for the strong RRQR algorithm and guarantees.
- Connection to low-rank approximation: Halko, N., Martinsson, P. G., and Tropp, J. A. *SIAM Review* 53(2), 2011, pp. 217-288, especially pp. 217-229 for the role of rank-revealing decompositions in low-rank approximation.

7. Quality control through SVD

Goal. Build a general quality-control module that runs exact SVD on small tasks and compares all matrix methods from Tasks 1-6 against it.

Input data. A dense matrix A , obtained directly or built from a functional matrix only for small m, n . Parameters: a list of ranks R , a list of methods, and tolerance.

Required implementation:

- Compute full or economy SVD: $A = U S V^T$.
- For each r , build the best rank- r approximation A_r by SVD.
- Compute the optimal tail errors $\|AA_r\|_F$ and $\|AA_r\|_2$.
- For each method from Tasks 2, 3, 5, and 6, compare its error with the SVD tail.
- For Task 1, compare approximate eigenvalues with exact values from `eigh` if A is symmetric.

Checks and experiments:

- Create a unified method-comparison table on several matrices.
- Show the ratio `error_method / error_svd_best` for the same r .
- Verify that the methods do not use SVD internally, except for this reference module.
- Test on matrices with different spectral decay patterns.

Submit:

- Function `benchmark_against_svd(A, methods, ranks)`.
- Tables and plots: error vs rank, time vs rank, singular values.
- A brief conclusion for each method: accuracy, speed, stability.

Acceptance criteria:

- SVD must be used only in reference mode.
- All methods must receive the same input data and the same ranks.
- The report must explicitly state that SVD is the benchmark for small tasks, but is not practical for large functional matrices.

Optional extensions:

- Add bootstrap/repeated runs for randomized methods.
- Add automatic saving of results to CSV/JSON.

5. Tensor algorithms

Reference pack for Task 7

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Eckart, C., and Young, G. "The approximation of one matrix by another of lower rank." *Psychometrika* 1(3), 1936, pp. 211-218. This is the exact reason the truncated SVD is the reference optimum for small dense tests.
- Practical SVD benchmarking: Halko, N., Martinsson, P. G., and Tropp, J. A. *SIAM Review* 53(2), 2011, pp. 217-288. Use pp. 217-229 for low-rank approximation goals, pp. 230-244 for algorithms that return U , S , V^T factors, and pp. 245-275 for interpreting Frobenius and spectral errors.
- Tensor-to-matrix notation support: Kolda, T. G., and Bader, B. W. "Tensor Decompositions and Applications." *SIAM Review* 51(3), 2009, pp. 455-500, especially pp. 455-462 for norms, matricization, and notation needed when the same quality-control ideas are later used for tensors.

8. ALS with a tensor in canonical decomposition form

Goal. Implement ALS for finding a CP approximation when the source tensor is already given in canonical CP format. The task is to obtain a CP decomposition of a specified target rank, possibly smaller than the original rank.

Input data. The source tensor X is given by CP factors of rank R_{source} : weights λ and factor matrices A_k . The goal is to find a CP representation of rank R_{target} . Parameters: R_{target} , max_iter , tol , regularization, and factor normalization.

Required implementation:

- Represent the target tensor Y in CP format with factor matrices $B^{(k)}$.
- At each ALS step, fix all factors except one and solve a linear problem to update the selected factor.
- Use the CP structure to compute Gram matrices and right-hand sides without constructing dense X , when possible.
- After each sweep, normalize factor columns and transfer scale factors into the weights.
- Add regularization for poorly conditioned normal equations.
- Compute the loss $\|XY\|^2$ through inner products of CP tensors.

Checks and experiments:

- Compress a CP tensor from rank R_{source} to smaller R_{target} and compare the error.
- Check reconstruction when R_{target} equals the true rank.
- Show error as a function of ALS iterations.
- Compare several random initializations.

Submit:

- Function `cp_als_from_cp(source_cp, rank, max_iter, tol, reg)`.
- A loss-vs-iteration plot and a table of final errors.
- A description of how the error is computed without a dense tensor.

Acceptance criteria:

- The algorithm must work with a CP source without mandatory densification.
- Factors after iterations must be normalized or their scale must be controlled.
- With multiple runs, the report must record the best and mean result.

Optional extensions:

- Add line search or damping for ALS.
- Add stopping by relative factor change.

Reference pack for Task 8

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Kolda, T. G., and Bader, B. W. "Tensor Decompositions and Applications." *SIAM Review* 51(3), 2009, pp. 455-500. Use pp. 455-462 for tensor notation, fibers, matricization, and products; pp. 463-475 for CP format, rank, uniqueness issues, normalization, and CP-ALS.
- Optimization and loss computation: Acar, E., Dunlavy, D. M., and Kolda, T. G. "A Scalable Optimization Approach for Fitting Canonical Tensor Decompositions." *Journal of Chemometrics* 25(2), 2011, pp. 67-86. Use pp. 67-72 for the CP objective and notation; pp. 73-80 for derivatives and optimization; pp. 81-86 for experiments and comparison with ALS.

9. ALS with a tensor in dense format

Goal. Implement classical CP-ALS for a dense tensor and compare it with the variant from Task 8.

Input data. Dense tensor X of size $n_1 \times \dots \times n_d$. Parameters: CP rank R , `max_iter`, `tol`, `reg`, and initialization method `random/svd`.

Required implementation:

- Implement unfolding/matricization for each mode.
- Implement the Khatri-Rao product for all factors except the one being updated.
- At each step, update factor $B^{(k)}$ by solving a least-squares problem.
- Normalize factors and store the weights λ .
- Compute $\text{fit} = 1 / \|XY\|_F / \|X\|_F$ after each sweep.
- Include regularization if the matrix of normal equations is poorly conditioned.

Checks and experiments:

- Generate a tensor of known CP rank and verify reconstruction.
- Add noise and examine how the optimal rank changes.
- Test different tensor sizes and orders: 3D and 4D.
- Compare random and SVD initialization.

Submit:

- Function `cp_als_dense(X, rank, init="random|svd", max_iter=...)`.
- Factors, weights, and fit/loss history.
- Report with a convergence plot and comments on local minima.

Acceptance criteria:

- With the correct rank and a good start, the error on an exact synthetic tensor must be small.
- Shapes of factor matrices must correspond to mode sizes.
- The code must work correctly for $d > 3$, not only for the three-dimensional case.

Optional extensions:

- Implement accelerated MTTKRP without explicitly building a huge Khatri-Rao product.
- Add nonnegative CP-ALS if all data are nonnegative.

Reference pack for Task 9

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Kolda, T. G., and Bader, B. W. SIAM Review 51(3), 2009, pp. 455-500. Use pp. 455-462 for dense tensor notation and unfoldings; pp. 463-475 for CP-ALS and Khatri-Rao/MTTKRP-style updates.
- Efficient tensor operations: Bader, B. W., and Kolda, T. G. "Efficient MATLAB Computations with Sparse and Factored Tensors." SIAM Journal on Scientific Computing 30(1), 2007/2008, pp. 205-231. Use pp. 205-214 for tensor storage and operations; pp. 215-224 for products needed in CP algorithms; pp. 225-231 for implementation examples.
- Algorithm comparison: Tomasi, G., and Bro, R. "A comparison of algorithms for fitting the PARAFAC model." Computational Statistics & Data Analysis 50(7), 2006, pp. 1700-1734. Use it for ALS behavior, acceleration, and practical failure modes.

10. Finding a canonical decomposition with a neural network

Goal. Formulate CP decomposition as differentiable optimization: factor matrices are trainable parameters, and reconstruction error is minimized by gradient methods.

Input data. Dense tensor X or functional/sample access to its elements. Parameters: CP rank R , optimizer Adam/LBFGS, `learning_rate`, `batch_size` for sample mode, and number of epochs.

Required implementation:

- Define trainable parameters: factor matrices $B^{(k)}$ and, when needed, weights λ .
- Implement forward(indices): compute $Y[i_1, \dots, i_d] = \sum_a \lambda_a \cdot \prod_k B_k[i_k, a]$.
- In dense mode, minimize MSE over all elements of X .
- For large tensors, minimize MSE over random batches of multi-indices.
- Add normalization or regularization of factors to avoid unbounded scale growth.
- Compare Adam and LBFGS or another second-order optimizer, if available.

Checks and experiments:

- On a tensor of known CP rank, verify recovery of factors up to permutation and scaling.

- Compare final error and runtime with CP-ALS from Task 9.
- Show the influence of `learning_rate` and `batch_size`.
- Check robustness to noise and overestimated rank.

Submit:

- Class `CPModel` or function `train_cp_neural(X_or_oracle, rank, ...)`.
- Training-loss and validation/sample-loss curves.
- Report on pros and cons of the neural-network approach versus ALS.

Acceptance criteria:

- The model must be able to compute values at arbitrary indices without building the full Y .
- The result must be reproducible for a fixed seed.
- The report must mention CP decomposition ambiguities: component permutation and factor scaling.

Optional extensions:

- Add early stopping by validation loss.
- Add mini-batch sampling with importance sampling over large elements.

Reference pack for Task 10

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Kolda, T. G., and Bader, B. W. *SIAM Review* 51(3), 2009, pp. 455-500. Use pp. 455-462 for tensors and pp. 463-475 for the CP model, scaling/permutation ambiguity, and ALS baseline.
- Differentiable optimization source: Acar, E., Dunlavy, D. M., and Kolda, T. G. *Journal of Chemometrics* 25(2), 2011, pp. 67-86. Use pp. 67-72 to formulate CP as an optimization problem; pp. 73-80 for derivatives and gradient-based solvers; pp. 81-86 for experiments and comparison against ALS.
- Stochastic/minibatch perspective: Sorber, L., Van Barel, M., and De Lathauwer, L. "Optimization-Based Algorithms for Tensor Decompositions." *SIAM Journal on Optimization* 23(2), 2013, pp. 695-720. Use pp. 695-704 for objective functions and parameterization; pp. 705-720 for gradient, nonlinear least-squares, and large-scale optimization perspectives.

11. Levenberg-Marquardt method for CP decomposition

Goal. Implement nonlinear least-squares optimization of CP factors by the Levenberg-Marquardt method and compare it with ALS and the gradient-based approach.

Input data. A dense tensor X of small or medium size. Desired CP rank R . Parameters: damping μ , rule for changing μ , `max_iter`, `tol`, and initial point from random initialization or ALS.

Required implementation:

- Write the parameter vector θ containing all factor matrices and weights.
- Define the residual vector $r(\theta) = \text{vec}(X - \text{CP}(\theta))$.
- Construct the Jacobian J analytically or through automatic differentiation.
- At each iteration, solve $(J^T J + \mu I) \delta = J^T r$ and update θ .
- If the loss decreases, accept the step and decrease μ ; otherwise reject the step and increase μ .
- Control factor-scaling ambiguity through normalization after the step.

Checks and experiments:

- Compare with ALS from identical initial approximations.
- Show number of iterations, runtime, and final loss.
- Test the method on small tensors where the explicit Jacobian fits in memory.
- Study the effect of the initial damping μ .

Submit:

- Function `cp levenberg_marquardt(X, rank, init, mu0, ...)`.
- History of loss and μ over iterations.
- Report: when LM converges faster and when it becomes too expensive.

Acceptance criteria:

- The method must correctly accept or reject steps.
- On small tasks, LM must decrease loss with a suitable μ setting.
- The report must state the number of parameters and the Jacobian dimensions.

Optional extensions:

- Use block or matrix-free Gauss-Newton without explicit J.
- Add bounds or factor regularization.

Reference pack for Task 11

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Tomasi, G., and Bro, R. "PARAFAC and missing values." *Chemometrics and Intelligent Laboratory Systems* 75(2), 2005, pp. 163-180. Use pp. 163-169 for PARAFAC/CP notation and residuals; pp. 170-180 for algorithmic handling of incomplete/residual-based least squares.
- Algorithm comparison: Tomasi, G., and Bro, R. "A comparison of algorithms for fitting the PARAFAC model." *Computational Statistics & Data Analysis* 50(7), 2006, pp. 1700-1734. Use pp. 1700-1711 for PARAFAC algorithms and pp. 1712-1734 for acceleration, convergence, and empirical comparison.
- Modern NLS/LM framework: Sorber, L., Van Barel, M., and De Lathauwer, L. "Optimization-Based Algorithms for Tensor Decompositions." *SIAM Journal on Optimization* 23(2), 2013, pp. 695-720. Use pp. 695-704 for CP parameterization and objective functions; pp. 705-720 for Gauss-Newton, Levenberg-Marquardt-style nonlinear least squares, and matrix-free ideas.

12. ST-HOSVD (Sequentially Truncated HOSVD)

Goal. Implement sequentially truncated HOSVD for constructing a Tucker approximation of a dense tensor with specified Tucker ranks or specified accuracy.

Input data. Dense tensor X of size $n_1 \times \dots \times n_d$. Parameters: ranks=($r_1, \dots, r_d$) or eps, mode processing order, and a rank-selection variant based on the tail of singular values.

Required implementation:

- Sequentially, for each mode k , build the unfolding of the current tensor.
- Compute the SVD of the unfolding and take the first r_k left singular vectors as factor U_k .
- Compress the current tensor by multiplying it by U_k^T along the corresponding mode.
- At the end, return the Tucker core G and factor matrices $U_1, \dots, U_d$.
- Implement `reconstruct(G, factors)` and `multilinear_rank`.
- For the eps variant, distribute the error budget across modes and choose r_k automatically.

Checks and experiments:

- Compare ST-HOSVD with ordinary HOSVD, where all SVDs are computed from the original X .
- Check reconstruction error and Tucker ranks.
- Generate a tensor of known Tucker rank and verify reconstruction.
- Show the effect of mode order on runtime and intermediate sizes.

Submit:

- Function `st_hosvd(X, ranks=None, eps=None, mode_order=None)`.
- Factors, core, and history of intermediate sizes and local errors.
- Report with table: ranks, rel_frob, storage, time.

Acceptance criteria:

- Core and factor shapes must strictly match the Tucker format.
- Factors U_k must be orthonormal.
- As ranks increase, the error must not increase.

Optional extensions:

- Add randomized SVD inside ST-HOSVD for large unfoldings.
- Make `mode_order` automatic based on current unfolding sizes.

Reference pack for Task 12

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Kolda, T. G., and Bader, B. W. *SIAM Review* 51(3), 2009, pp. 455-500. Use pp. 455-462 for tensor notation and mode products; pp. 476-478 for Tucker and HOSVD; pp. 479-500 for applications and extensions.
- Classical HOSVD source: De Lathauwer, L., De Moor, B., and Vandewalle, J. "A Multilinear Singular Value Decomposition." *SIAM Journal on Matrix Analysis and Applications* 21(4), 2000, pp. 1253-1278. Use pp. 1253-1261 for multilinear SVD definitions; pp. 1262-1278 for properties, truncation, and interpretation.
- ST-HOSVD source: Vannieuwenhoven, N., Vandebril, R., and Meerbergen, K. "A New Truncation Strategy for the Higher-Order Singular Value Decomposition." *SIAM Journal on Scientific Computing* 34(2), 2012, pp. A1027-A1052. Use pp. A1027-A1036 for

error expressions; pp. A1037-A1046 for sequential truncation; pp. A1047-A1052 for numerical experiments.

13. TT-cross for functional tensors

Goal. Implement or carefully prototype TT-cross: construction of a tensor-train approximation of a multidimensional functional tensor using a small number of element queries.

Input data. Functional tensor X of size $n_1 \times \dots \times n_d$, available through entry/batch. Parameters: eps , max_rank or fixed TT ranks, max_sweeps , and initial multi-indices.

Required implementation:

- Define the TT format: cores G_k of size $r_{k-1} \times n_k \times r_k$, where $r_0 = r_d = 1$.
- Organize the selection of cross indices for unfoldings between the left and right groups of modes.
- Use `maxvol`/greedy pivot selection to choose informative indices.
- Compute only the required tensor elements through the oracle; do not build the full X .
- Implement error estimation on a random sample of multi-indices.
- Create at least a working version for small d and fixed TT ranks; present adaptive rank as an extension.

Checks and experiments:

- Check on a tensor that is already specified in TT format with known rank.
- Check on an analytical functional tensor, for example $\exp(\sum_k x_k^2)$ or $1/(1+\sum_k i_k)$.
- Compare TT storage against dense storage.
- Show sample error versus the number of oracle calls.
- On small sizes, compare with TT-SVD or dense reconstruction.

Submit:

- Function `tt_cross(Tfun, eps=None, max_rank=None, ranks=None, ...)`.
- Class TT with methods `entry(indices)`, `reconstruct_small()`, and `storage()`.
- Log of ranks, selected pivots, `oracle_calls`, and `sample_error`.

Acceptance criteria:

- The full tensor must not be built inside the algorithm for large tests.
- TT cores must have consistent dimensions r_{k-1} , n_k , r_k .
- On an exact TT test, the error must be small with sufficient ranks.

Optional extensions:

- Add adaptive TT-rank selection.
- Add TT-rounding after TT-cross.
- Compare different strategies for initial indices.

Reference pack for Task 13

Use these sources in order. Together they cover the task from zero-knowledge background to algorithm design, tests, and implementation pitfalls.

- Zero-knowledge path: Kolda, T. G., and Bader, B. W. SIAM Review 51(3), 2009, pp. 455-500, especially pp. 455-462 for tensor notation, multi-indices, unfoldings, and norms.
- Tensor-train basics: Oseledets, I. V. "Tensor-Train Decomposition." SIAM Journal on Scientific Computing 33(5), 2011, pp. 2295-2317. Use pp. 2295-2303 for TT format and storage; pp. 2304-2312 for TT-SVD and algebra; pp. 2313-2317 for complexity and examples.
- TT-cross source: Oseledets, I. V., and Tyrtshnikov, E. E. "TT-cross approximation for multidimensional arrays." Linear Algebra and its Applications 432(1), 2010, pp. 70-88. Use pp. 70-76 for the black-box tensor setting; pp. 77-82 for TT interpolation/cross construction and `maxvol`; pp. 83-88 for oracle-call complexity and experiments.

6. Recommended repository structure

```
project/
  README.md
  requirements.txt or environment.yml
  src/
    functional_matrix.py
    matrix_algorithms/
      lanczos.py
      cross.py
      randomized_svd.py
      recompression.py
      pivoted_qr.py
      svd_control.py
    tensor_algorithms/
      cp_als.py
      cp_neural.py
      cp_lm.py
      st_hosvd.py
      tt_cross.py
  tests/
    test_matrix_algorithms.py
    test_tensor_algorithms.py
  experiments/
    run_matrix_benchmarks.py
    run_tensor_benchmarks.py
  reports/
    figures/
    results.csv
    report.pdf or report.md
```

- README.md: how to install dependencies, run tests, and reproduce experiments.
- src/: clean algorithm implementations without heavy experiments inside.
- tests/: fast unit tests that run in minutes.
- experiments/: scripts for tables and figures in the report.
- reports/: final results, figures, and written conclusions.

7. Minimum contents of the final report

- A brief statement of each implemented task.
- A description of input data and algorithm parameters.
- Pseudocode or a flowchart for nontrivial parts.
- Tables with errors, runtime, ranks, and the number of oracle calls.
- Convergence plots for iterative methods: Lanczos, ALS, neural network, and LM.
- Comparison with SVD for matrix tasks on small problem sizes.
- Analysis of failure cases: poor conditioning, slow spectral decay, local minima, overestimated or underestimated rank.
- Conclusions: which method is better for which type of data.

8. Pre-submission checklist

All 13 tasks have separate functions/classes, or the report explicitly states which tasks were not implemented and why.

Tasks 2, 3, and 4 really work with FunctionalMatrix and do not build the full matrix inside the algorithm.

Each algorithm has at least one automated test.

Matrix methods have common SVD-based quality control on small problem sizes.

Tensor methods have a test on a synthetic tensor of known rank.

The report contains tables with rel_frob/sample_rmse, runtime, rank, and oracle calls.

All randomized experiments are reproducible through seed.

The code does not crash on invalid parameters, but returns a clear error or warning.

README allows a new person to run the project without oral explanations.

9. Short glossary of terms

Term	Meaning
Functional matrix	a matrix whose elements are computed by a function; the full array is not stored in advance.
Low-rank approximation	approximation of an object by a sum of a small number of simple components.
Cross approximation	approximation of a matrix/tensor through informative rows, columns, or slices.
Recompression	reduction of redundant rank in an already constructed decomposition with error control.
CP/canonical format	representation of a tensor as a sum of rank-1 outer products.
Tucker/ST-HOSVD	representation of a tensor through a small core and orthonormal factor matrices by modes.
TT/tensor train	chain representation of a multidimensional tensor through three-dimensional cores.
Oracle calls	number of real calls to the function that computes elements of a matrix or tensor.